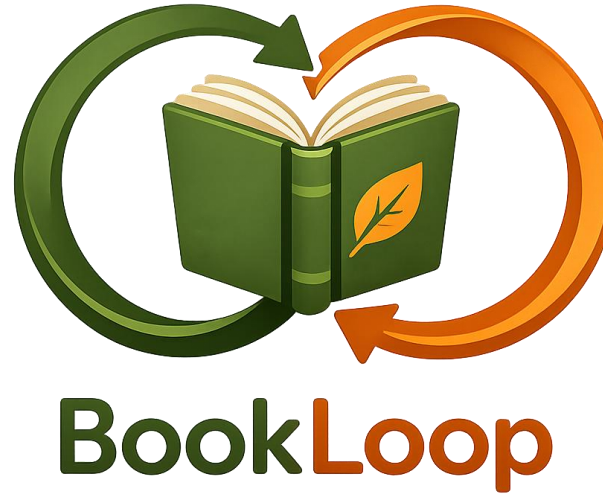




Implementierung einer relationalen Datenbank für eine Buchtausch-App

Erarbeitungs-/Reflexionsphase

Brian Pudelko
Matrikelnummer 4252311
Informatik · DLBDSPBDM01_D

Entität: Rolle

SQL-Statement

```
CREATE TABLE rolle (  
  rolle_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  rollenname VARCHAR(50) NOT NULL UNIQUE,  
  beschreibung VARCHAR(255)  
);
```

Testfall

```
2  SELECT  
3      rolle_id,  
4      rollenname,  
5      beschreibung  
6  FROM rolle  
7  ORDER BY rolle_id;
```

Data Output			
Messages			
Notifications			
        SQL			
	rolle_id [PK] integer	rollenname character varying (50)	beschreibung character varying (255)
1	1	Benutzer	Standardrolle für registrierte Benutzer
2	2	Administrator	Verwaltet Benutzer, Stammdaten und Plattfo...
3	3	Moderator	Modertiert Bewertungen und Kommentare
4	4	Support	Unterstützt Benutzer bei Problemen
5	5	Redakteur	Pflegt Inhalte und Stammdaten
6	6	Pruefer	Besitzt lesenden Zugriff für Prüfzwecke
7	7	Gast	Besitzt eingeschränkten Zugriff
8	8	Testbenutzer	Rolle für interne Funktionstests
9	9	Datenpfleger	Pflegt Datenbestände der Plattform
10	10	Systemverwaltung	Verwaltet technische Systemeinstellungen

Entität: Benutzer

SQL-Statement

```
1 CREATE TABLE benutzer (  
2     benutzer_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
3     rolle_id INTEGER NOT NULL,  
4     vorname VARCHAR(100) NOT NULL,  
5     nachname VARCHAR(100) NOT NULL,  
6     email VARCHAR(255) NOT NULL UNIQUE,  
7     telefon VARCHAR(30),  
8     passwort_hash VARCHAR(255) NOT NULL,  
9     registrierungsdatum TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
10    aktiv BOOLEAN NOT NULL DEFAULT TRUE,  
11  
12    CONSTRAINT fk_benutzer_rolle  
13        FOREIGN KEY (rolle_id)  
14        REFERENCES rolle(rolle_id)  
15 );
```

Testfall

```
1 SELECT  
2     b.benutzer_id,  
3     b.vorname,  
4     b.nachname,  
5     b.email,  
6     b.aktiv,  
7     r.rollenname  
8 FROM benutzer b  
9 JOIN rolle r  
10     ON b.rolle_id = r.rolle_id  
11 ORDER BY b.benutzer_id;
```

	benutzer_id integer	vorname character varying (100)	nachname character varying (100)	email character varying (255)	aktiv boolean	rollenname character varying (50)
1	1	Anna	Schmidt	anna.schmidt@example.de	true	Benutzer
2	2	Lukas	Weber	lukas.weber@example.de	true	Benutzer
3	3	Sophie	Mueller	sophie.mueller@example.de	true	Benutzer
4	4	Jonas	Fischer	jonas.fischer@example.de	true	Benutzer
5	5	Laura	Wagner	laura.wagner@example.de	true	Benutzer
6	6	Felix	Becker	felix.becker@example.de	true	Benutzer
7	7	Marie	Hoffmann	marie.hoffmann@example....	true	Benutzer
8	8	David	Koch	david.koch@example.de	true	Benutzer
9	9	Julia	Richter	julia.richter@example.de	true	Benutzer
10	10	Max	Keller	max.keller@example.de	true	Administrator

Entität: Standort

SQL-Statement

```
1 CREATE TABLE standort (  
2     standort_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
3     benutzer_id INTEGER NOT NULL,  
4     bezeichnung VARCHAR(100),  
5     strasse VARCHAR(150) NOT NULL,  
6     hausnummer VARCHAR(20) NOT NULL,  
7     plz VARCHAR(10) NOT NULL,  
8     ort VARCHAR(100) NOT NULL,  
9     land VARCHAR(100) NOT NULL DEFAULT 'Deutschland',  
10    breitengrad DECIMAL(9,6),  
11    laengengrad DECIMAL(9,6),  
12    ist_standard BOOLEAN NOT NULL DEFAULT FALSE,  
13  
14    CONSTRAINT fk_standort_benutzer  
15        FOREIGN KEY (benutzer_id)  
16        REFERENCES benutzer(benutzer_id)  
17 );
```

Testfall

```
1 SELECT  
2     s.standort_id,  
3     b.vorname,  
4     b.nachname,  
5     s.bezeichnung,  
6     s.strasse,  
7     s.hausnummer,  
8     s.plz,  
9     s.ort,  
10    s.land,  
11    s.ist_standard  
12 FROM standort s  
13 JOIN benutzer b USING (benutzer_id)  
14 ORDER BY s.standort_id;
```

	standort_id integer	vorname character varying (100)	nachname character varying (100)	bezeichnung character varying (100)	strasse character varying (150)	hausnummer character varying (20)	plz character varying (10)	ort character varying (100)	land character varying (100)	ist_standard boolean
1	1	Anna	Schmidt	Zuhause	Musterstrasse	12	72070	Tuebingen	Deutschland	true
2	2	Lukas	Weber	Zuhause	Gartenstrasse	8	72074	Tuebingen	Deutschland	true
3	3	Sophie	Mueller	Zuhause	Bahnhofstrasse	21	72764	Reutlingen	Deutschland	true
4	4	Jonas	Fischer	Zuhause	Hauptstrasse	45	72108	Rottenburg	Deutschland	true
5	5	Laura	Wagner	Zuhause	Kirchstrasse	7	72072	Tuebingen	Deutschland	true
6	6	Felix	Becker	Zuhause	Schillerstrasse	18	72762	Tuebingen	Deutschland	true
7	7	Marie	Hoffmann	Zuhause	Neckarstrasse	30	72072	Tuebingen	Deutschland	true
8	8	David	Koch	Zuhause	Umlandstrasse	14	72108	Rottenburg	Deutschland	true
9	9	Julia	Richter	Zuhause	Lindenstrasse	5	72076	Tuebingen	Deutschland	true
10	10	Anna	Schmidt	Arbeitsplatz	Wilhelmstrasse	25	72074	Tuebingen	Deutschland	false

Entität: Autor

SQL-Statement

```
1 CREATE TABLE autor (  
2     autor_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
3     vorname VARCHAR(100),  
4     nachname VARCHAR(100) NOT NULL  
5 );
```

Testfall

```
1 SELECT  
2     autor_id,  
3     vorname,  
4     nachname  
5 FROM autor  
6 ORDER BY autor_id;
```

Data Output Messages Notifications

Showing rows: 1 to 10 Page No: 1 of 1

	autor_id [PK] integer	vorname character varying (100)	nachname character varying (100)
1	1	J. K.	Rowling
2	2	George	Orwell
3	3	J. R. R.	Tolkien
4	4	Jane	Austen
5	5	Franz	Kafka
6	6	Hermann	Hesse
7	7	Agatha	Christie
8	8	Stephen	King
9	9	Cornelia	Funke
10	10	Michael	Ende

Entität: Verlag

SQL-Statement

```
1 CREATE TABLE verlag (  
2     verlag_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
3     verlagsname VARCHAR(150) NOT NULL UNIQUE  
4 );
```

Testfall

```
1 SELECT  
2     verlag_id,  
3     verlagsname  
4 FROM verlag  
5 ORDER BY verlag_id;
```

Data Output Messages Notifications

Showing rows: 1 to 10 Page No: 1

	verlag_id [PK] integer	verlagsname character varying (150)
1	1	Carlsen Verlag
2	2	Fischer Verlag
3	3	Rowohlt Verlag
4	4	Suhrkamp Verlag
5	5	Piper Verlag
6	6	Heyne Verlag
7	7	Klett-Cotta
8	8	dtv Verlagsgesellschaft
9	9	Penguin Verlag
10	10	Bastei Luebbe

Entität: Genre

SQL-Statement

```
1 CREATE TABLE genre (  
2     genre_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
3     bezeichnung VARCHAR(100) NOT NULL UNIQUE  
4 );
```

Testfall

```
1 SELECT  
2     genre_id,  
3     bezeichnung  
4 FROM genre  
5 ORDER BY genre_id;
```

Data Output Messages Notifications

Showing rows: 1 to 10 Page No: 1

	genre_id [PK] integer	bezeichnung character varying (100)
1	1	Fantasy
2	2	Science-Fiction
3	3	Krimi
4	4	Thriller
5	5	Roman
6	6	Historischer Roman
7	7	Sachbuch
8	8	Biografie
9	9	Kinderbuch
10	10	Klassiker

Entität: Sprache

SQL-Statement

```
1 CREATE TABLE sprache (  
2     sprache_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
3     bezeichnung VARCHAR(100) NOT NULL UNIQUE  
4 );
```

Testfall

```
1 SELECT  
2     sprache_id,  
3     bezeichnung  
4 FROM sprache  
5 ORDER BY sprache_id;
```

Data Output Messages Notifications

Showing rows: 1 to 10 Page No: 1

	sprache_id [PK] integer	bezeichnung character varying (100)
1	1	Deutsch
2	2	Englisch
3	3	Franzoesisch
4	4	Spanisch
5	5	Italienisch
6	6	Portugiesisch
7	7	Norwegisch
8	8	Schwedisch
9	9	Chinesisch
10	10	Japanisch

Entität: Buch

SQL-Statement

```
1 CREATE TABLE buch (  
2     buch_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
3     autor_id INTEGER NOT NULL,  
4     verlag_id INTEGER,  
5     genre_id INTEGER NOT NULL,  
6     sprache_id INTEGER NOT NULL,  
7     titel VARCHAR(255) NOT NULL,  
8     isbn VARCHAR(20) UNIQUE,  
9     erscheinungsjahr INTEGER,  
10    beschreibung TEXT,  
11  
12    CONSTRAINT fk_buch_autor  
13        FOREIGN KEY (autor_id)  
14        REFERENCES autor(autor_id),  
15  
16    CONSTRAINT fk_buch_verlag  
17        FOREIGN KEY (verlag_id)  
18        REFERENCES verlag(verlag_id),  
19  
20    CONSTRAINT fk_buch_genre  
21        FOREIGN KEY (genre_id)  
22        REFERENCES genre(genre_id),  
23  
24    CONSTRAINT fk_buch_sprache  
25        FOREIGN KEY (sprache_id)  
26        REFERENCES sprache(sprache_id)  
27 );
```

Testfall

```
1 SELECT  
2     b.buch_id,  
3     b.titel,  
4     a.vorname,  
5     a.nachname,  
6     v.verlagsname,  
7     g.bezeichnung AS genre,  
8     s.bezeichnung AS sprache,  
9     b.erscheinungsjahr  
10 FROM buch b  
11 JOIN autor a USING (autor_id)  
12 LEFT JOIN verlag v USING (verlag_id)  
13 JOIN genre g USING (genre_id)  
14 JOIN sprache s USING (sprache_id)  
15 ORDER BY b.buch_id;
```

	buch_id integer	titel character varying (255)	vorname character varying (100)	nachname character varying (100)	verlagsname character varying (150)	genre character varying (100)	sprache character varying (100)	erscheinungsjahr integer
1	1	Harry Potter und der Stein der Weis...	J. K.	Rowling	Carlsen Verlag	Fantasy	Deutsch	1998
2	2	1984	George	Orwell	Fischer Verlag	Klassiker	Deutsch	1949
3	3	Der Herr der Ringe	J. R. R.	Tolkien	Klett-Cotta	Fantasy	Deutsch	1954
4	4	Stolz und Vorurteil	Jane	Austen	Rowohlt Verlag	Klassiker	Deutsch	1813
5	5	Die Verwandlung	Franz	Kafka	Suhrkamp Verlag	Klassiker	Deutsch	1915
6	6	Der Steppenwolf	Hermann	Hesse	Piper Verlag	Roman	Deutsch	1927
7	7	Mord im Orient-Express	Agatha	Christie	Heyne Verlag	Krimi	Deutsch	1934
8	8	Es	Stephen	King	Penguin Verlag	Thriller	Deutsch	1986
9	9	Tintenherz	Cornelia	Funke	Carlsen Verlag	Kinderbuch	Deutsch	2003
10	10	Die unendliche Geschichte	Michael	Ende	dtv Verlagsgesellschaft	Kinderbuch	Deutsch	1979

Entität: Zustand

SQL-Statement

```
1 CREATE TABLE zustand (  
2     zustand_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
3     bezeichnung VARCHAR(100) NOT NULL UNIQUE,  
4     beschreibung VARCHAR(255)  
5 );
```

Testfall

```
1 SELECT  
2     zustand_id,  
3     bezeichnung,  
4     beschreibung  
5 FROM zustand  
6 ORDER BY zustand_id;
```

Data Output Messages Notifications

	zustand_id [PK] integer	bezeichnung character varying (100)	beschreibung character varying (255)
1	1	Neu	Unbenutztes Buch in neuwertigem Zustand
2	2	Wie neu	Kaum Gebrauchsspuren, nahezu neuwertig
3	3	Sehr gut	Leichte Gebrauchsspuren, insgesamt sehr guter Zustand
4	4	Gut	Normale Gebrauchsspuren ohne größere Schäden
5	5	Akzeptabel	Deutlich sichtbare Gebrauchsspuren, aber vollständig nutz...
6	6	Gebraucht	Mehrere sichtbare Gebrauchsspuren vorhanden
7	7	Mit Markierungen	Enthält Markierungen oder Unterstreichungen
8	8	Mit Notizen	Enthält handschriftliche Notizen
9	9	Beschädigter Einband	Einband weist sichtbare Beschädigungen auf
10	10	Sammlerstück	Besonderes oder älteres Exemplar mit Sammlerwert

Entität: Angebot

SQL-Statement

```
1 CREATE TYPE anbotstatus AS ENUM (  
2     'verfuegbar',  
3     'reserviert',  
4     'ausgeliehen',  
5     'deaktiviert'  
6 );  
7  
8 CREATE TABLE anbot (  
9     anbot_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
10    anbieter_id INTEGER NOT NULL,  
11    buch_id INTEGER NOT NULL,  
12    zustand_id INTEGER NOT NULL,  
13    standort_id INTEGER,  
14    maximale_leihdauer_tage INTEGER NOT NULL,  
15    abholung_moeglich BOOLEAN NOT NULL DEFAULT TRUE,  
16    versand_moeglich BOOLEAN NOT NULL DEFAULT FALSE,  
17    versandkosten DECIMAL(8,2),  
18    status anbotstatus NOT NULL DEFAULT 'verfuegbar',  
19    einstelldatum TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
20    bemerkung TEXT,  
21  
22    CONSTRAINT ck_anbot_leihdauer  
23        CHECK (maximale_leihdauer_tage > 0),  
24  
25    CONSTRAINT fk_anbot_anbieter  
26        FOREIGN KEY (anbieter_id)  
27        REFERENCES benutzer(benutzer_id),  
28  
29    CONSTRAINT fk_anbot_buch  
30        FOREIGN KEY (buch_id)  
31        REFERENCES buch(buch_id),  
32  
33    CONSTRAINT fk_anbot_zustand  
34        FOREIGN KEY (zustand_id)  
35        REFERENCES zustand(zustand_id),  
36  
37    CONSTRAINT fk_anbot_standort  
38        FOREIGN KEY (standort_id)  
39        REFERENCES standort(standort_id)  
40 );
```

Testfall

```
1 SELECT  
2     a.anbot_id,  
3     b.vorname,  
4     b.nachname,  
5     bu.titel,  
6     z.bezeichnung AS zustand,  
7     s.ort,  
8     a.maximale_leihdauer_tage,  
9     a.versand_moeglich,  
10    a.status  
11 FROM anbot a  
12 JOIN benutzer b  
13     ON a.anbieter_id = b.benutzer_id  
14 JOIN buch bu USING (buch_id)  
15 JOIN zustand z USING (zustand_id)  
16 LEFT JOIN standort s USING (standort_id)  
17 ORDER BY a.anbot_id;
```

anbot_id	integer	vorname	character varying (100)	nachname	character varying (100)	titel	character varying (255)	zustand	character varying (100)	ort	character varying (100)	maximale_leihdauer_tage	integer	versand_moeglich	boolean	status	anbotstatus
1		Anna		Schmidt		Harry Potter und der Stein der Weis...		Wie neu		Tuebingen		21		false		verfuegbar	
2		Lukas		Weber		1984		Sehr gut		Tuebingen		14		true		verfuegbar	
3		Sophie		Mueller		Der Herr der Ringe		Gut		Reutlingen		28		false		verfuegbar	
4		Jonas		Fischer		Stolz und Vorurteil		Wie neu		Rottenburg		21		true		verfuegbar	
5		Laura		Wagner		Die Verwandlung		Akzeptabel		Tuebingen		14		false		verfuegbar	
6		Felix		Becker		Der Steppenwolf		Sehr gut		Reutlingen		28		true		verfuegbar	
7		Marie		Hoffmann		Mord im Orient-Express		Wie neu		Tuebingen		21		false		verfuegbar	
8		David		Koch		Es		Gut		Rottenburg		14		true		verfuegbar	
9		Julia		Richter		Tintenherz		Sehr gut		Tuebingen		28		false		verfuegbar	
10		Anna		Schmidt		Die unendliche Geschichte		Gut		Tuebingen		21		true		verfuegbar	

Entität: Verfügbarkeit

SQL-Statement

```
CREATE TABLE verfuegbarkeit (  
    verfuegbarkeit_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    anbot_id INTEGER NOT NULL,  
    beginn TIMESTAMP NOT NULL,  
    ende TIMESTAMP NOT NULL,  
    verfuegbar BOOLEAN NOT NULL DEFAULT TRUE,  
  
    CONSTRAINT ck_verfuegbarkeit_zeitraum  
        CHECK (ende > beginn),  
  
    CONSTRAINT fk_verfuegbarkeit_anbot  
        FOREIGN KEY (anbot_id)  
        REFERENCES anbot(anbot_id)  
);
```

Testfall

```
1 SELECT  
2     v.verfuegbarkeit_id,  
3     v.anbot_id,  
4     b.titel,  
5     v.beginn,  
6     v.ende,  
7     v.verfuegbar  
8 FROM verfuegbarkeit v  
9 JOIN anbot a USING (anbot_id)  
10 JOIN buch b USING (buch_id)  
11 ORDER BY v.verfuegbarkeit_id;
```

	verfuegbarkeit_id	anbot_id	titel	beginn	ende	verfuegbar
	integer	integer	character varying (255)	timestamp without time zone	timestamp without time zone	boolean
1	1	1	Harry Potter und der Stein der Weis...	2026-09-01 09:00:00	2026-09-01 18:00:00	true
2	2	2	1984	2026-09-02 10:00:00	2026-09-02 17:00:00	true
3	3	3	Der Herr der Ringe	2026-09-03 08:00:00	2026-09-03 16:00:00	true
4	4	4	Stolz und Vorurteil	2026-09-04 12:00:00	2026-09-04 19:00:00	true
5	5	5	Die Verwandlung	2026-09-05 09:30:00	2026-09-05 15:30:00	true
6	6	6	Der Steppenwolf	2026-09-06 11:00:00	2026-09-06 18:00:00	true
7	7	7	Mord im Orient-Express	2026-09-07 14:00:00	2026-09-07 20:00:00	true
8	8	8	Es	2026-09-08 09:00:00	2026-09-08 17:00:00	true
9	9	9	Tintenherz	2026-09-09 10:30:00	2026-09-09 18:30:00	true
10	10	10	Die unendliche Geschichte	2026-09-10 08:30:00	2026-09-10 16:30:00	true

Entität: Reservierung

SQL-Statement

```
1 CREATE TYPE reservierungsstatus AS ENUM (  
2     'aktiv',  
3     'storniert',  
4     'abgelaufen',  
5     'angenommen'  
6 );  
7  
8 CREATE TABLE reservierung (  
9     reservierung_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
10    benutzer_id INTEGER NOT NULL,  
11    anbot_id INTEGER NOT NULL,  
12    reservierungsdatum TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
13    gueltig_bis TIMESTAMP NOT NULL,  
14    status reservierungsstatus NOT NULL DEFAULT 'aktiv',  
15  
16    CONSTRAINT fk_reservierung_benutzer  
17        FOREIGN KEY (benutzer_id)  
18        REFERENCES benutzer(benutzer_id),  
19  
20    CONSTRAINT fk_reservierung_anbot  
21        FOREIGN KEY (anbot_id)  
22        REFERENCES anbot(anbot_id)  
23 );
```

Testfall

```
1 SELECT  
2     r.reservierung_id,  
3     b.vorname,  
4     b.nachname,  
5     r.anbot_id,  
6     bu.titel,  
7     r.reservierungsdatum,  
8     r.gueltig_bis,  
9     r.status  
10 FROM reservierung r  
11 JOIN benutzer b USING (benutzer_id)  
12 JOIN anbot a USING (anbot_id)  
13 JOIN buch bu USING (buch_id)  
14 ORDER BY r.reservierung_id;
```

Data Output Messages Notifications

	reservierung_id integer	vorname character varying (100)	nachname character varying (100)	anbot_id integer	titel character varying (255)	reservierungsdatum timestamp without time zone	gueltig_bis timestamp without time zone	status reservierungsstatus
1	1	Lukas	Weber	1	Harry Potter und der Stein der Weis...	2026-08-01 10:00:00	2026-08-03 10:00:00	angenommen
2	2	Sophie	Mueller	2	1984	2026-08-02 11:30:00	2026-08-04 11:30:00	angenommen
3	3	Jonas	Fischer	3	Der Herr der Ringe	2026-08-03 09:15:00	2026-08-05 09:15:00	angenommen
4	4	Laura	Wagner	4	Stolz und Vorurteil	2026-08-04 14:00:00	2026-08-06 14:00:00	angenommen
5	5	Felix	Becker	5	Die Verwandlung	2026-08-05 16:20:00	2026-08-07 16:20:00	angenommen
6	6	Marie	Hoffmann	6	Der Steppenwolf	2026-08-06 12:45:00	2026-08-08 12:45:00	angenommen
7	7	David	Koch	7	Mord im Orient-Express	2026-08-23 08:30:00	2026-08-26 08:30:00	aktiv
8	8	Julia	Richter	8	Es	2026-08-23 13:10:00	2026-08-27 13:10:00	aktiv
9	9	Max	Keller	9	Tintenherz	2026-08-12 15:00:00	2026-08-14 15:00:00	storniert
10	10	Lukas	Weber	10	Die unendliche Geschichte	2026-08-10 17:30:00	2026-08-12 17:30:00	abgelaufen

Entität: Reservierung

Trigger

```
CREATE OR REPLACE FUNCTION pruefe_angebot_verfuegbar()
RETURNS TRIGGER AS $$
BEGIN
    IF (
        SELECT status
        FROM anbot
        WHERE anbot_id = NEW.anbot_id
    ) <> 'verfuegbar' THEN

        RAISE EXCEPTION
            'Das Angebot % ist nicht verfuegbar und kann nicht reserviert werden.',
            NEW.anbot_id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_reservierung_anbot_verfuegbar
BEFORE INSERT OR UPDATE OF anbot_id
ON reservierung
FOR EACH ROW
EXECUTE FUNCTION pruefe_angebot_verfuegbar();
```

Negativtest

```
Query | Query History
1      UPDATE anbot
2      SET status = 'reserviert'
3      WHERE anbot_id = 1;
```

```
1      INSERT INTO reservierung
2      (benutzer_id, anbot_id, gueltig_bis, status)
3      VALUES
4      (3, 1, '2026-09-15 18:00:00', 'aktiv');
```

Data Output | Messages | Notifications

ERROR: Das Angebot 1 ist nicht verfuegbar und kann nicht reserviert werden.
CONTEXT: PL/pgSQL function pruefe_angebot_verfuegbar() line 9 at RAISE

SQL state: P0001

```
Query | Query History
1      UPDATE anbot
2      SET status = 'verfuegbar'
3      WHERE anbot_id = 1;
```

Entität: Ausleihe

SQL-Statement

```
1 CREATE TYPE ausleihstatus AS ENUM (  
2     'reserviert',  
3     'aktiv',  
4     'zurueckgegeben',  
5     'ueberfaellig',  
6     'storniert'  
7 );  
8  
9 CREATE TABLE ausleihe (  
10     ausleihe_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
11     ausleiher_id INTEGER NOT NULL,  
12     anbot_id INTEGER NOT NULL,  
13     reservierung_id INTEGER UNIQUE,  
14     ausleihdatum DATE NOT NULL,  
15     faelligkeitsdatum DATE NOT NULL,  
16     rueckgabedatum DATE,  
17     versand BOOLEAN NOT NULL DEFAULT FALSE,  
18     status ausleihstatus NOT NULL DEFAULT 'aktiv',  
19  
20     CONSTRAINT fk_ausleihe_benutzer  
21         FOREIGN KEY (ausleiher_id)  
22         REFERENCES benutzer(benutzer_id),  
23  
24     CONSTRAINT fk_ausleihe_anbot  
25         FOREIGN KEY (anbot_id)  
26         REFERENCES anbot(anbot_id),  
27  
28     CONSTRAINT fk_ausleihe_reservierung  
29         FOREIGN KEY (reservierung_id)  
30         REFERENCES reservierung(reservierung_id)  
31 );
```

Testfall

```
1 SELECT  
2     a.ausleihe_id,  
3     b.vorname,  
4     b.nachname,  
5     a.anbot_id,  
6     bu.titel,  
7     a.reservierung_id,  
8     a.ausleihdatum,  
9     a.faelligkeitsdatum,  
10    a.rueckgabedatum,  
11    a.versand,  
12    a.status  
13 FROM ausleihe a  
14 JOIN benutzer b  
15     ON a.ausleiher_id = b.benutzer_id  
16 JOIN anbot an USING (anbot_id)  
17 JOIN buch bu USING (buch_id)  
18 LEFT JOIN reservierung r USING (reservierung_id)  
19 ORDER BY a.ausleihe_id;
```

	ausleihe_id integer	vorname character varying (100)	nachname character varying (100)	anbot_id integer	titel character varying (255)	reservierung_id integer	ausleihdatum date	faelligkeitsdatum date	rueckgabedatum date	versand boolean	status ausleihstatus
1	1	Lukas	Weber	1	Harry Potter und der Stein der Weis...	1	2026-08-03	2026-08-24	2026-08-22	false	zurueckgegeb...
2	2	Sophie	Mueller	2	1984	2	2026-08-04	2026-08-18	[null]	true	ueberfaellig
3	3	Jonas	Fischer	3	Der Herr der Ringe	3	2026-08-05	2026-09-02	[null]	false	aktiv
4	4	Laura	Wagner	4	Stolz und Vorurteil	4	2026-08-06	2026-08-27	[null]	true	aktiv
5	5	Felix	Becker	5	Die Verwandlung	5	2026-08-07	2026-08-21	2026-08-20	false	zurueckgegeb...
6	6	Marie	Hoffmann	6	Der Steppenwolf	6	2026-08-08	2026-09-05	[null]	true	aktiv
7	7	David	Koch	7	Mord im Orient-Express	[null]	2026-08-18	2026-09-08	[null]	false	aktiv
8	8	Julia	Richter	8	Es	[null]	2026-08-19	2026-09-02	[null]	true	aktiv
9	9	Max	Keller	9	Tintenherz	[null]	2026-08-20	2026-09-17	[null]	false	aktiv
10	10	Lukas	Weber	10	Die unendliche Geschichte	[null]	2026-08-21	2026-09-11	[null]	true	aktiv

Entität: Bewertung

SQL-Statement

```
1 CREATE TABLE bewertung (  
2     bewertung_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
3     ausleihe_id INTEGER NOT NULL UNIQUE,  
4     sterne INTEGER NOT NULL,  
5     kommentar TEXT,  
6     bewertungsdatum TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
7  
8     CONSTRAINT ck_bewertung_sterne  
9         CHECK (sterne BETWEEN 1 AND 5),  
10  
11     CONSTRAINT fk_bewertung_ausleihe  
12         FOREIGN KEY (ausleihe_id)  
13         REFERENCES ausleihe(ausleihe_id)  
14 );
```

Testfall

```
1 SELECT  
2     bw.bewertung_id,  
3     bw.ausleihe_id,  
4     b.vorname,  
5     b.nachname,  
6     bu.titel,  
7     bw.sterne,  
8     bw.kommentar,  
9     bw.bewertungsdatum  
10 FROM bewertung bw  
11 JOIN ausleihe a USING (ausleihe_id)  
12 JOIN benutzer b  
13     ON a.ausleiher_id = b.benutzer_id  
14 JOIN angebot an USING (angebot_id)  
15 JOIN buch bu USING (buch_id)  
16 ORDER BY bw.bewertung_id;
```

	bewertung_id integer 🔒	ausleihe_id integer 🔒	vorname character varying (100) 🔒	nachname character varying (100) 🔒	titel character varying (255) 🔒	sterne integer 🔒	kommentar text 🔒	bewertungsdatum timestamp without time zone 🔒
1	1	1	Lukas	Weber	Harry Potter und der Stein der Weis...	5	Sehr guter Zustand und unkomplizierte Überg...	2026-08-24 16:36:47.532911
2	2	2	Sophie	Mueller	1984	4	Buch entsprach der Beschreibung	2026-08-24 16:36:47.532911
3	3	3	Jonas	Fischer	Der Herr der Ringe	5	Sehr angenehme Ausleihe	2026-08-24 16:36:47.532911
4	4	4	Laura	Wagner	Stolz und Vorurteil	4	Schnelle und einfache Abwicklung	2026-08-24 16:36:47.532911
5	5	5	Felix	Becker	Die Verwandlung	5	Buch war in sehr gutem Zustand	2026-08-24 16:36:47.532911
6	6	6	Marie	Hoffmann	Der Steppenwolf	3	Ausleihe insgesamt zufriedenstellend	2026-08-24 16:36:47.532911
7	7	7	David	Koch	Mord im Orient-Express	4	Unkomplizierte Übergabe	2026-08-24 16:36:47.532911
8	8	8	Julia	Richter	Es	5	Sehr gute Erfahrung	2026-08-24 16:36:47.532911
9	9	9	Max	Keller	Tintenherz	4	Alles wie beschrieben	2026-08-24 16:36:47.532911
10	10	10	Lukas	Weber	Die unendliche Geschichte	5	Problemlose Ausleihe und Rückgabe	2026-08-24 16:36:47.532911

Constraint-Negativtests : PRIMARY KEY | FOREIGN KEY

```
1 INSERT INTO rolle
2   (rolle_id, rollenname, beschreibung)
3   OVERRIDING SYSTEM VALUE
4   VALUES
5   (1, 'Testrolle', 'Negativtest für Primärschlüssel');
```

Data Output Messages Notifications

ERROR: duplicate key value violates unique constraint "rolle_pkey"
Key (rolle_id)=(1) already exists.

SQL state: 23505
Detail: Key (rolle_id)=(1) already exists.

```
Query Query History
1 INSERT INTO benutzer
2   (rolle_id, vorname, nachname, email, passwort_hash)
3   VALUES
4   (9999, 'Test', 'Fremdschlüssel', 'fk-test@bookloop.de', 'testhash');
```

Data Output Messages Notifications

ERROR: insert or update on table "benutzer" violates foreign key constraint "fk_benutzer_rolle"
Key (rolle_id)=(9999) is not present in table "rolle".

SQL state: 23503
Detail: Key (rolle_id)=(9999) is not present in table "rolle".

Constraint-Negativtests : UNIQUE | ENUM

```
1 INSERT INTO benutzer
2     (rolle_id, vorname, nachname, email, passwort_hash)
3 VALUES
4     (2, 'Test', 'Unique', 'anna.schmidt@example.de', 'testhash');
```

Data Output Messages Notifications

ERROR: duplicate key value violates unique constraint "benutzer_email_key"
Key (email)=(anna.schmidt@example.de) already exists.

SQL state: 23505

Detail: Key (email)=(anna.schmidt@example.de) already exists.

```
1 UPDATE anbot
2 SET status = 'kaputt'
3 WHERE anbot_id = 1;
```

Data Output Messages Notifications

ERROR: invalid input value for enum anbotstatus: "kaputt"

LINE 2: SET status = 'kaputt'

^

SQL state: 22P02

Character: 29

Komplexe Abfragen und Aggregationen

Aktivste Nutzer

```
SELECT
  b.benutzer_id,
  b.vorname,
  b.nachname,
  COUNT(a.ausleihe_id) AS anzahl_ausleihen
FROM benutzer b
LEFT JOIN ausleihe a
  ON b.benutzer_id = a.ausleiher_id
GROUP BY
  b.benutzer_id,
  b.vorname,
  b.nachname
ORDER BY
  anzahl_ausleihen DESC,
  b.benutzer_id;
```

Showing rows: 1 to 10 | Page No: 1 | of 1

	benutzer_id [PK] integer	vorname character varying (100)	nachname character varying (100)	anzahl_ausleihen bigint
1	2	Lukas	Weber	2
2	3	Sophie	Mueller	1
3	4	Jonas	Fischer	1
4	5	Laura	Wagner	1
5	6	Felix	Becker	1
6	7	Marie	Hoffmann	1
7	8	David	Koch	1
8	9	Julia	Richter	1
9	10	Max	Keller	1
10	1	Anna	Schmidt	0

Beliebte Genre

```
SELECT
  g.genre_id,
  g.bezeichnung AS genre,
  COUNT(a.ausleihe_id) AS anzahl_ausleihen
FROM genre g
LEFT JOIN buch b
  USING (genre_id)
LEFT JOIN anbot an
  USING (buch_id)
LEFT JOIN ausleihe a
  USING (angebot_id)
GROUP BY
  g.genre_id,
  g.bezeichnung
ORDER BY
  anzahl_ausleihen DESC,
  g.genre_id;
```

	genre_id [PK] integer	genre character varying (100)	anzahl_ausleihen bigint
1	10	Klassiker	3
2	1	Fantasy	2
3	9	Kinderbuch	2
4	3	Krimi	1
5	4	Thriller	1
6	5	Roman	1
7	2	Science-Fiction	0
8	6	Historischer Roman	0
9	7	Sachbuch	0
10	8	Biografie	0

Performance und Indexanalyse

```
SELECT
  tablename,
  indexname,
  indexdef
FROM pg_indexes
WHERE schemaname = 'public'
ORDER BY tablename, indexname;
```

	tablename name	indexname name	indexdef text
1	angebot	angebot_pkey	CREATE UNIQUE INDEX anbot_pkey ON public.anbot USING btree (anbot_id)
2	ausleihe	ausleihe_pkey	CREATE UNIQUE INDEX ausleihe_pkey ON public.ausleihe USING btree (ausleihe_id)
3	ausleihe	ausleihe_reservierung_id_k...	CREATE UNIQUE INDEX ausleihe_reservierung_id_key ON public.ausleihe USING btree (reservierun...
4	autor	autor_pkey	CREATE UNIQUE INDEX autor_pkey ON public.autor USING btree (autor_id)
5	benutzer	benutzer_email_key	CREATE UNIQUE INDEX benutzer_email_key ON public.benutzer USING btree (email)
6	benutzer	benutzer_pkey	CREATE UNIQUE INDEX benutzer_pkey ON public.benutzer USING btree (benutzer_id)
7	bewertung	bewertung_ausleihe_id_key	CREATE UNIQUE INDEX bewertung_ausleihe_id_key ON public.bewertung USING btree (ausleihe_id)
8	bewertung	bewertung_pkey	CREATE UNIQUE INDEX bewertung_pkey ON public.bewertung USING btree (bewertung_id)
9	buch	buch_isbn_key	CREATE UNIQUE INDEX buch_isbn_key ON public.buch USING btree (isbn)
10	buch	buch_pkey	CREATE UNIQUE INDEX buch_pkey ON public.buch USING btree (buch_id)
11	genre	genre_bezeichnung_key	CREATE UNIQUE INDEX genre_bezeichnung_key ON public.genre USING btree (bezeichnung)
12	genre	genre_pkey	CREATE UNIQUE INDEX genre_pkey ON public.genre USING btree (genre_id)
13	reservierung	reservierung_pkey	CREATE UNIQUE INDEX reservierung_pkey ON public.reservierung USING btree (reservierung_id)
14	rolle	rolle_pkey	CREATE UNIQUE INDEX rolle_pkey ON public.rolle USING btree (rolle_id)
15	rolle	rolle_rollenname_key	CREATE UNIQUE INDEX rolle_rollenname_key ON public.rolle USING btree (rollenname)
16	sprache	sprache_bezeichnung_key	CREATE UNIQUE INDEX sprache_bezeichnung_key ON public.sprache USING btree (bezeichnung)
17	sprache	sprache_pkey	CREATE UNIQUE INDEX sprache_pkey ON public.sprache USING btree (sprache_id)
18	standort	standort_pkey	CREATE UNIQUE INDEX standort_pkey ON public.standort USING btree (standort_id)
19	verfuegbark...	verfuegbarkeit_pkey	CREATE UNIQUE INDEX verfuegbarkeit_pkey ON public.verfuegbarkeit USING btree (verfuegbarkeit...
20	verlag	verlag_pkey	CREATE UNIQUE INDEX verlag_pkey ON public.verlag USING btree (verlag_id)
21	verlag	verlag_verlagsname_key	CREATE UNIQUE INDEX verlag_verlagsname_key ON public.verlag USING btree (verlagsname)
22	zustand	zustand_bezeichnung_key	CREATE UNIQUE INDEX zustand_bezeichnung_key ON public.zustand USING btree (bezeichnung)
23	zustand	zustand_pkey	CREATE UNIQUE INDEX zustand_pkey ON public.zustand USING btree (zustand_id)

Query Query History

```
1 CREATE INDEX idx_angebot_buch_id
2 ON anbot (buch_id);
3
4 CREATE INDEX idx_angebot_anbieter_id
5 ON anbot (anbieter_id);
6
7 CREATE INDEX idx_reservierung_angebot_id
8 ON reservierung (angebot_id);
9
10 CREATE INDEX idx_ausleihe_angebot_id
11 ON ausleihe (angebot_id);
I
```

Data Output Messages Notifications

CREATE INDEX

Query returned successfully in 73 msec.

23	verfuegbark...	verfuegbarkeit_pkey	CREATE UNIQUE INDEX verfuegbarkeit_pkey ON public.verfuegbarkeit USING btree (verfuegbarkeit...
24	verlag	verlag_pkey	CREATE UNIQUE INDEX verlag_pkey ON public.verlag USING btree (verlag_id)
25	verlag	verlag_verlagsname_key	CREATE UNIQUE INDEX verlag_verlagsname_key ON public.verlag USING btree (verlagsname)
26	zustand	zustand_bezeichnung_key	CREATE UNIQUE INDEX zustand_bezeichnung_key ON public.zustand USING btree (bezeichnung)
27	zustand	zustand_pkey	CREATE UNIQUE INDEX zustand_pkey ON public.zustand USING btree (zustand_id)